
ORM AND OODB

Presenters

Bivhas Devkota

Dindayal Neupane

Dinesh Gurung

Oshan Bajracharya

Paras Pun

Sworup RajBhandari

Sudeep Mishra

19th January

OBJECT-RELATIONAL MAPPING (ORM)

- **ORM is a programming technique that facilitates the interaction between an OOP language (like Python, Java, or C#) and a relational database (like MySQL, PostgreSQL, or SQLite).**
- **Its main purpose is that it translates object-oriented data structures into relational tables and vice versa.**

KEY FEATURES OF ORM

- **Abstraction**
 - Developers work with objects and classes, not SQL queries.
 - **Transaction Management**
 - ORM tools handle database transactions automatically.
 - **Schema Mapping**
 - Maps database schemas to object models seamlessly.
 - **Lazy Loading**
 - Loads data from the database only when it's accessed, improving performance.
-

HOW DOES ORM WORK ?

➤ **Mapping Objects to Tables**

- Classes in the object-oriented language are mapped to tables in the database.
- Each class corresponds to a table, and each instance of the class represents a row.

➤ **Attributes to Columns**

- Class attributes (fields) map to the columns of the table.

➤ **Automatic SQL Generation**

- ORM tools automatically generate SQL queries for CRUD (Create, Read, Update, Delete) operations.

➤ **Handling Relationships**

- Relationships between objects (like one-to-one, one-to-many, many-to-many) are managed using foreign keys and join tables.

➤ **Abstraction**

Object in OOP
(e.g., User Object)

- **Name**
- **Email**

Relational DB
(e.g., User Table)

- **Name (Column)**
- **Email (Column)**

ORM Framework

- **Maps Properties to Columns**
- **Maps Object to Table**
- **Converts OOP Calls to SQL Queries**

ADVANTAGES OF ORM

- **Faster Development**
 - Focus on the application logic rather than database interactions.
 - **Database Independence**
 - Switch databases without changing the application code.
 - **Consistency**
 - ORM ensures the database and object models stay synchronized.
 - **Code Maintainability**
 - Changes to database schemas or models are easier to manage.
-

DISADVANTAGES OF ORM

- **Performance Overhead**
 - ORM adds a layer of abstraction, which can slow down performance for complex queries.
 - **Complexity for Advanced Queries**
 - Writing custom queries for intricate requirements can be cumbersome.
 - **Abstraction Leaks**
 - When the ORM-generated queries do not align well with the database's capabilities.
-

COMMON USE CASES OF ORM

- **Web Applications**
 - Frameworks like Django and Ruby on Rails rely heavily on ORM for managing databases.
 - **Enterprise Applications**
 - Simplifies the development of large-scale systems requiring frequent database interactions.
 - **E-commerce Platforms**
 - Enables quick development of systems for inventory, user data, and transactions.
-

OBJECT ORIENTED DATABASE (OODB)

- **DBMS that integrates the principles of Object-oriented Programming (OOP) into its design and operation.**
 - **Stores data as objects, just like we do in OOP languages like C++, Java, Python etc.**
 - **Object is something that represents both data (what it is) and actions (what it can do).
For example:**
 - **A Car object in programming might have:**
 - *Data (Attributes): Color, brand, engine type etc.**
 - *Actions (Methods): Start, stop, honk etc.**
-

KEY FEATURES OF AN OODB

➤ **Object Storage**

- Data is stored as objects, which includes:

Attributes (Properties): Information about the object

Methods (Behavior): Functions or actions the object can perform

➤ **Support for Object-Oriented Concepts**

- **Classes and Instances:** Templates for objects (i.e., blueprint for creating objects). Objects are instances of classes.
 - **Inheritance:** Classes can inherit properties and behaviors from other classes.
 - **Polymorphism:** Objects can process tasks differently based on their data type.
 - **Encapsulation:** Data and behaviors are bundled together.
-

➤ **Complex Data Types**

- Supports storing complex and hierarchical data (e.g., nested objects or arrays within objects).

➤ **Object Identity**

- Each object has unique Object Identifier (OID), used to retrieve it without relying on primary keys or indices. (e.g., a fingerprint which can be a unique ID for an object).
-

HOW DOES AN OODB WORK?

➤ **Object Persistence**

- Objects created in an application are saved to the database, and can be saved directly in the database without converting them into tables and rows like in traditional databases.

➤ **Querying**

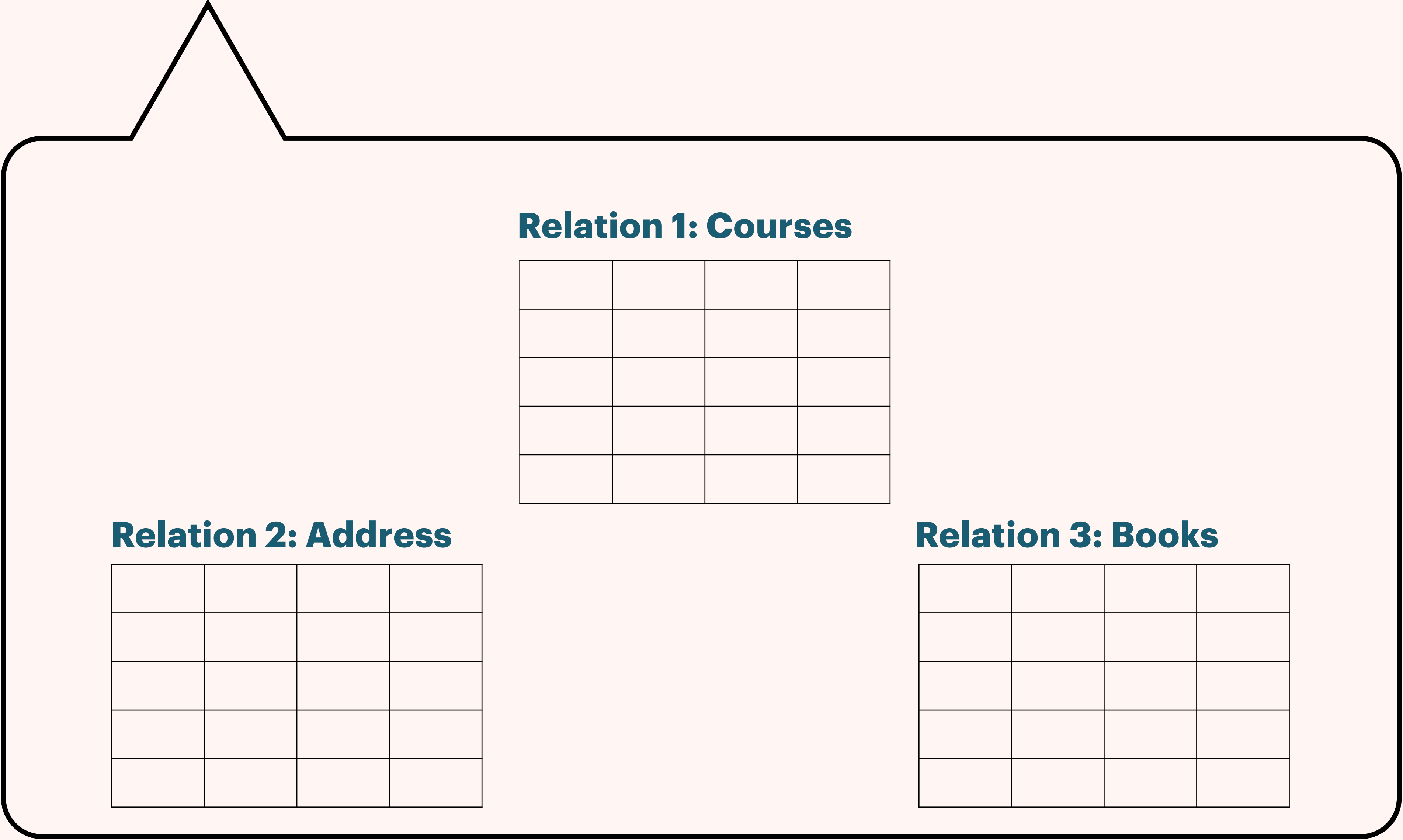
- Uses an Object Query Language (OQL) or integrates seamlessly with programming language queries (e.g., Java or Python methods).

➤ **Interfacing with OOP Languages**

- If one object is related to another, these relationships are stored naturally. Directly maps database objects to programming objects, avoiding the need for Object-Relational Mapping (ORM).

➤ **Built-in relationships and features**

Object: Student



Relation 1: Courses

Relation 2: Address

Relation 3: Books

ADVANTAGES OF OODB

- **Seamless Integration with OOP**
 - **Supports Complex Data: Efficiently handles nested or hierarchical data.**
 - **Improved Performance**
 - **Reduced Impedance Mismatch: Eliminates the conceptual gap between the database model (objects) and the application model (objects).**
-

DISADVANTAGES OF OODB

- **Limited Adoption:** Not as widely used or supported as relational database.
 - **Learning Curve:** Requires knowledge of both OOP and OODB principles.
 - **Lack of Standardization:** Fewer standard tools and protocols compared to relational databases (like SQL).
 - **Query Complexity:** Querying objects can sometimes be more complex than SQL queries for relational data.
-

USE CASES FOR OODBS

- **Engineering Applications: CAD/CAM systems for complex modeling**
- **Simulation System**
- **Multimedia Applications**
- **Real-Time Systems**

Example of OODBs: db40 (A lightweight OODB for Java and .NET), ObjectDB (Designed specifically for Java application), Versant Object Database (High-performance OODB for enterprise systems)

COMPARISON WITH RELATIONAL DATABASES

Feature	Relational Database	Object-Oriented Database
Data Model	Tables, rows, columns	Objects, attributes, methods
Query Language	SQL	OQL or integrated with OOP
Relationships	Primary and foreign keys	Direct object links
Performance	Better for tabular data	Better for hierarchical/ complex data
Standardization	Highly standardized (SQL-based)	Limited

ORM VS OODB

ORM	OODB
Data stored in relational tables (rows and columns)	Data stored as objects
Manipulate objects, but SQL is generated behind the scenes	Directly manipulate objects (no SQL needed)
May have some overhead due to automatic SQL generation	Typically faster as it don't need object-relational mapping

CONCLUSION

ANY QUERIES ?

Thank You
